Object Oriented Programming (Class Relationships)

December 26, 2023

Table of contents

1. Introduction
2. Association
3. Aggregation
4. Composition

Association

- If two classes in a model need to communicate with each other, there must be a link between them. This link can be represented by an association.
- Association can be represented by a line between these classes with an arrow indicating the navigation direction. In case an arrow is on both sides, the association is known as a bidirectional association.
- Note that an association between two objects is not the same thing as inheritance between two classes. Association means that one object uses another object or a function in that other object.
- Inheritance is said to be an is-a relationship whereas association is known as a has-a relationship. There may be one-to-one, one-to-many, many-to-one, and many-to-many associations between objects.
- In association, both of the classes are not dependent on each other. They can exist independently. If one is destroyed, it doesn't mean the other one also has to be destroyed.

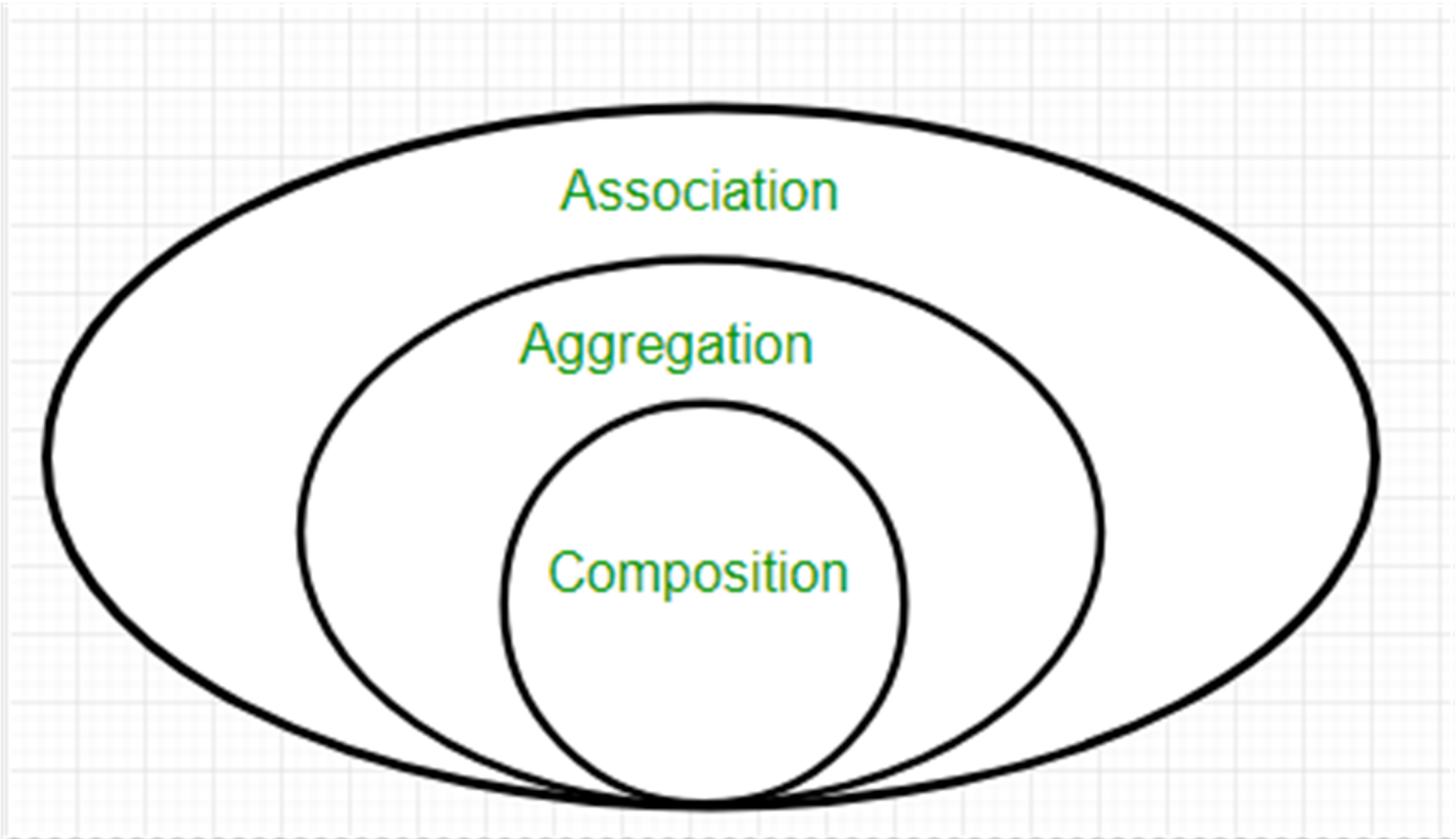
Aggregation

- Aggregation is a stronger form of association in which a container object has reference to other objects.
- Both of the classes are not dependent on each other. They can exist independently. If one is destroyed, it doesn't mean the other one also has to be destroyed.
- Consider the example of a car and a driver. A car can exist without a driver and a driver can exist without the car. If we destroy a car, the driver might still survive and vice versa. Hence they are not dependant on each other. The lifetime of both is independent of each other.
- An aggregation is denoted as a solid line with an unfilled diamond at the end. The diamond is connected to the class that represents the aggregate.
- Aggregation relationships may be bidirectional.

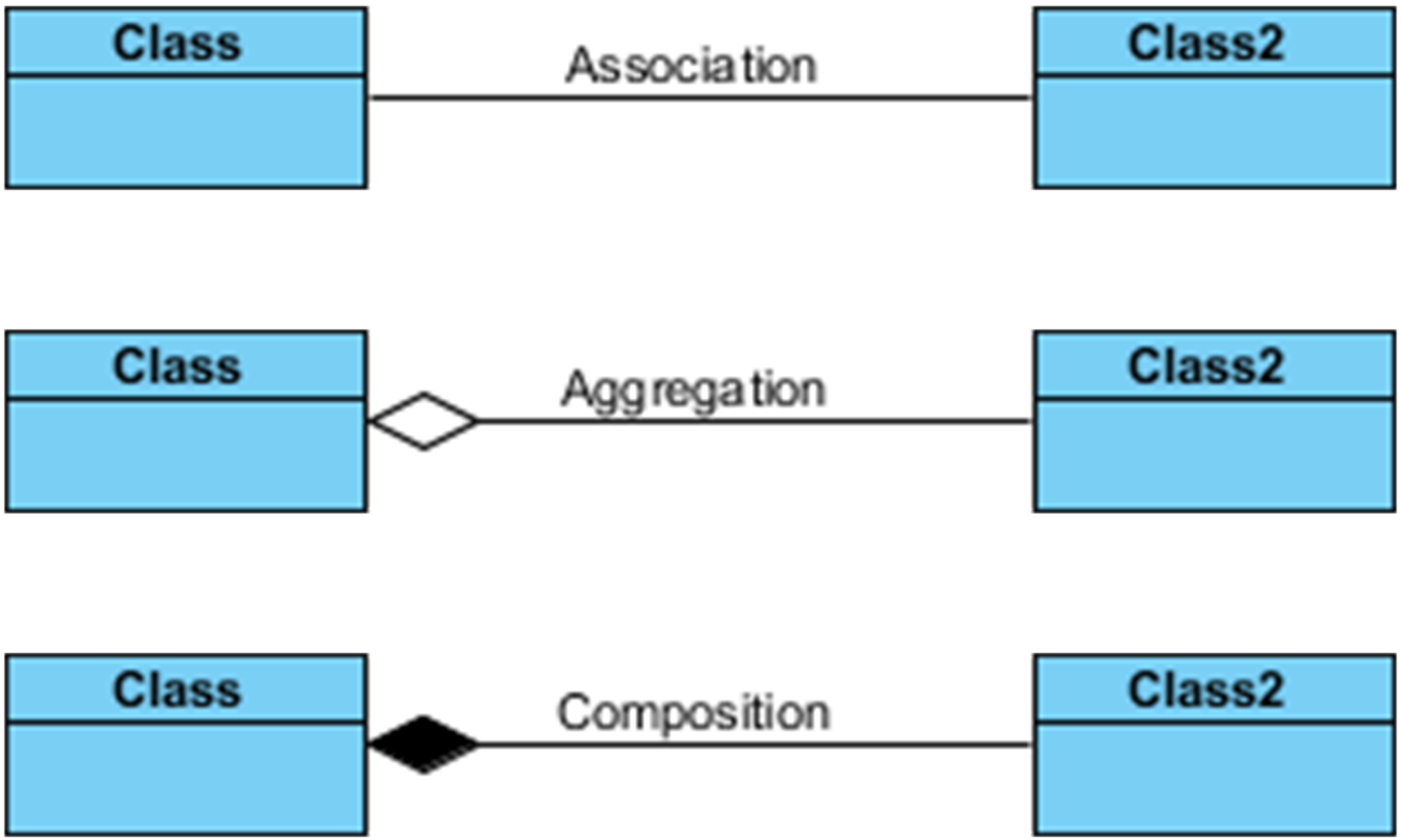
Composition

- Composition is a stronger form of aggregation that occurs when an object's life is tightly bound to another object's life.
- When the main object dies (i.e., is deleted), all the objects that are associated with that object also die. This means that the referenced object is solely contained by the referring object.
- Consider the example of a car and its engine. A car cannot exist without an engine and an engine is useless without a car. If we destroy a car, its engine will also be destroyed. Hence they are dependant on each other. The lifetime of both is dependent of each other.
- A composition is denoted as a solid line with an filled diamond at the end. The diamond is connected to the class that represents the composite (whole).
- In a composition relationship, data usually flows in only one direction (i.e., from the whole class to the part class).

Pictorial Representation



UML Representation



Example 1

```
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printA(){cout<<" I am Class A \n";}
};
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printBA(A obj)
     {cout<<" Class B <uses> Class A:";obj.printA();}
    void printB(){cout<<" I am Class B \n";}
};

int main()   {
   A a;
   {
     B b; b.printBA(a);
     b.printB();
   }
   a.printA();
return 0;
}
```

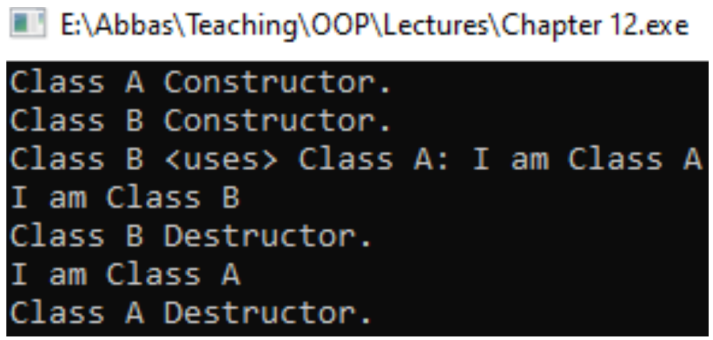
E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class A Constructor.
Class B Constructor.
Class B <uses> Class A: I am Class A
Class A Destructor.
I am Class B
Class B Destructor.
I am Class A
Class A Destructor.
```

Example 2

```
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printA(){cout<<" I am Class A \n";}
};
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printBA(A& obj)
     {cout<<" Class B <uses> Class A:";obj.printA();}
    void printB(){cout<<" I am Class B \n";}
};

int main()  {
   A a;
   {
     B b; b.printBA(a);
     b.printB();
   }
   a.printA();
return 0;
}
```



Example 3

```
class B;    // Forward declaration
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printAB(B); // cannot be defined here
    void printA(){cout<<" I am Class A \n";}
 };
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printB(){cout<<" I am Class B \n";}
};
void A::printAB(B obj)
  {cout<<" Class A <uses> Class B:";obj.printB();}
int main()   {
    B b;
    {
      A a; a.printAB(b);
      a.printA();
    }
    b.printB();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class B Constructor.
Class A Constructor.
Class A <uses> Class B: I am Class B
Class B Destructor.
I am Class A
Class A Destructor.
I am Class B
Class B Destructor.
```

Example 4

```
class B;    // Forward declaration
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printAB(B&); // Forward declaration
    void printA(){cout<<" I am Class A \n";}
 };
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printB(){cout<<" I am Class B \n";}
};
void A::printAB(B& obj)
  {cout<<" Class A <uses> Class B:";obj.printB();}
int main()  {
   B b;
   {
    A a; a.printAB(b);
    a.printA();
   }
   b.printB();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class B Constructor.
Class A Constructor.
Class A <uses> Class B: I am Class B
I am Class A
Class A Destructor.
I am Class B
Class B Destructor.
```

Example 5

```
class B; // Forward declaration
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printAB(B); // Forward declaration
    void printA(){cout<<" I am Class A \n";}
 };
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printB(){cout<<" I am Class B \n";}
    void printBA(A obj)
     {cout<<" Class B <uses> Class A:";obj.printA();}
 };
void A::printAB(B obj) {cout<<" Class A <uses> Class B:";obj.printB();}
int main()  {
   B b;
    {
    A a;a.printA();
    a.printAB(b);
    b.printBA(a);
    }
   b.printB();
return 0;
}
```

```
Class B Constructor.
Class A Constructor.
I am Class A
Class A <uses> Class B: I am Class B
Class B Destructor.
Class B <uses> Class A: I am Class A
Class A Destructor.
Class A Destructor.
I am Class B
Class B Destructor.
```

Example 6

```
class B; // Forward declaration
class A{
  public:
    A(){cout<<" Class A Constructor.\n";}
    ~A(){cout<<" Class A Destructor.\n";}
    void printAB(B&); // Forward declaration
    void printA(){cout<<" I am Class A \n";}
 };
class B{
  public:
    B(){cout<<" Class B Constructor.\n";}
    ~B(){cout<<" Class B Destructor.\n";}
    void printB(){cout<<" I am Class B \n";}
    void printBA(A& obj)
     {cout<<" Class B <uses> Class A:";obj.printA();}
 };
void A::printAB(B& obj) {cout<<" Class A <uses> Class B:";obj.printB();}
int main()  {
   B b;
    {
    A a;a.printA();
    a.printAB(b);
    b.printBA(a);
    }
   b.printB();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class B Constructor.
Class A Constructor.
I am Class A
Class A <uses> Class B: I am Class B
Class B <uses> Class A: I am Class A
Class A Destructor.
I am Class B
Class B Destructor.
```

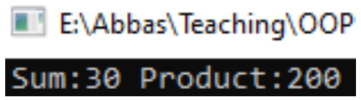
Exercise

```
// Does the following code represent an association
// between class A and class B?
class B;
class A{
    int a;
  public:
    A(int x){a = x;}
    void sum(B&);
    int getA(){return a;}
};

class B{
    int b;
  public:
    B(int x){b = x;}
    void product(A &obj){cout<<" Product:"<< b*(obj.getA());}
    int getB(){return b;}
 };
void A:: sum(B& obj){cout<<" Sum:"<< a+(obj.getB());}

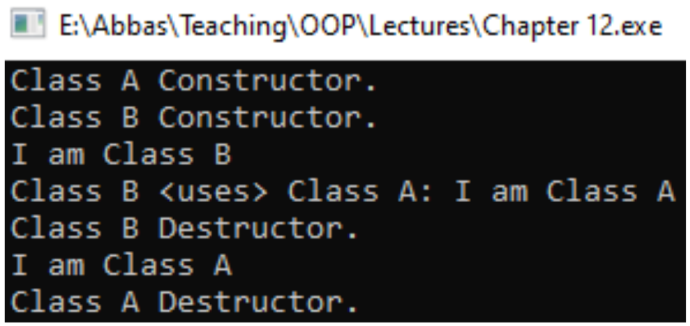
int main()  {
   B b(10); A a(20); a.sum(b); b.product(a);

 return 0;
}
```



Example 1

```
class A{
  public:
   A(){cout<<" Class A Constructor.\n";}
   ~A(){cout<<" Class A Destructor.\n";}
   void printA(){cout<<" I am Class A \n";}
};
class B{
    A* ap;
  public:
    B(A* x){cout<<" Class B Constructor.\n"; ap = x;}
    ~B(){cout<<" Class B Destructor.\n";}
    void printBA()
     {cout<<" Class B <uses> Class A:";ap->printA();}
    void printB(){cout<<" I am Class B \n";}
 };
int main()  {
    A a;
    {
    B b (&a);
    b.printB();
    b.printBA();
    }
    a.printA();
return 0;
}
```



Example 2

```
class B;   // Forward declaration
class A{
    B* bp;
  public:
    A(B* x){cout<<" Class A Constructor.\n"; bp = x;}
    ~A(){cout<<" Class A Destructor.\n";}
    void printA(){cout<<" I am Class A \n";}
    void printAB();   // Forward declaration
};
class B{
  public:
    B(){cout<<" Class B Constructor.\n"; }
    ~B(){cout<<" Class B Destructor.\n";}
    void printB(){cout<<" I am Class B \n";}
};
void A::printAB()
  {cout<<" Class A <uses> Class B:";bp->printB();}
int main()   {
    B b;
    {
     A a (&b);
     a.printA();
     a.printAB();
    }
    b.printB();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class B Constructor.
Class A Constructor.
I am Class A
Class A <uses> Class B: I am Class B
Class A Destructor.
I am Class B
Class B Destructor.
```

Example 3

```
class B;
class A{
   B* bp;
  public:
   A(B* x){cout<<" Class A Constructor.\n"; bp = x;}
   ~A(){cout<<" Class A Destructor.\n";}
   void printA(){cout<<" I am Class A \n";}
   void printAB();
};
class B{
   A* ap;
  public:
   B(A* x){cout<<" Class B Constructor.\n"; ap = x;}
   ~B(){cout<<" Class B Destructor.\n";}
   void printBA(){cout<<" Class B <uses> Class A:";ap->printA();}
   void printB(){cout<<" I am Class B \n";}
};
void A::printAB(){cout<<" Class A <uses> Class B:";bp->printB();}
int main()  {
   A a(NULL);
   {
       B b(&a); b.printB();
       b.printBA();
   }
   a.printA(); a.printAB();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class A Constructor.
Class B Constructor.
I am Class B
Class B <uses> Class A: I am Class A
Class B Destructor.
I am Class A
Class A <uses> Class B: I am Class B
Class A Destructor.
```

Example 4

```
class B;
class A{
   B* bp;
  public:
   A(B* x){cout<<" Class A Constructor.\n"; bp = x;}
   ~A(){cout<<" Class A Destructor.\n";}
   void printA(){cout<<" I am Class A \n";}
   void printAB();
};
class B{
   A* ap;
  public:
   B(A* x){cout<<" Class B Constructor.\n"; ap = x;}
   ~B(){cout<<" Class B Destructor.\n";}
   void printBA(){cout<<" Class B <uses> Class A:";ap->printA();}
   void printB(){cout<<" I am Class B \n";}
};
void A::printAB(){cout<<" Class A <uses> Class B:";bp->printB();}
int main()  {
   B b(NULL);
   {
    A a(&b); a.printA();
    a.printAB();
   }
   b.printB();b.printBA();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe

```
Class B Constructor.
Class A Constructor.
I am Class A
Class A <uses> Class B: I am Class B
Class A Destructor.
I am Class B
Class B <uses> Class A: I am Class A
Class B Destructor.
```

Example 5

```
class Car{
    int cModel; string cName;
 public:
    Car(string n,int m){cModel = m; cName = n;}
    void carInfo()
      {cout<<" Car Name: "<<cName<<", Model: "<<cModel<<endl;}
};
class Driver{
    string dName;
    Car* cptr;
 public:
    Driver(string name, Car* c) {dName = name, cptr = c;}
    void Info()
      {cout<<" Driver Name: "<<dName; cptr->carInfo();}
};
int main(){
   Car toyota("Passo", 2012);
   {
    Driver owner ("Zakria,", &toyota);
    owner.Info();
   }
   toyota.carInfo();
return 0;
}
```

```
E:\Abbas\Teaching\OOP\Lectures\Chapter 12.exe
Driver Name: Zakria, Car Name: Passo, Model: 2012
Car Name: Passo, Model: 2012
```

Example 6

```
class Driver;
class Car{
    Driver* dptr; int cModel; string cName;
 public:
    Car(string n,int m, Driver* d){cModel = m; cName = n; dptr = d;}
    void Info(); //Forward declaration
};
class Driver{
    string dName;
 public:
    Driver(string name) {dName = name;}
    void driverInfo(){cout<<", Driver Name: "<<dName;}
};
void Car::Info(){
      cout<<" Car Name: "<<cName<<", Model: "<<cModel;
      dptr->driverInfo();
    }
int main(){
  Driver owner ("Zakria");
  {
   Car toyota("Passo", 2012,&owner);
   toyota.Info();
  }
  owner.driverInfo();
return 0;
}
```

```
Car Name: Passo, Model: 2012, Driver Name: Zakria, Driver Name: Zakria
```

Composition

- A class can have objects of other classes as data members. This capability is called composition. Composition is also referred to as a has-a relationship.
- Member objects are constructed in the order in which they are declared (not in the order in which they are listed in the member *initializer – list*).
- They are constructed before their enclosing objects.
- If a class member is an object of another class, making that member object *public* does not violate the encapsulation and hiding of that member object's *private* members. But, it does violate the encapsulation and hiding of the containing class's implementation, so member objects of class types should still be *private*, like all other data members.
- If a member object is not initialized through a member initializer, the member object's default constructor will be called implicitly. Values, if any, established by the default constructor can be overridden by *set* functions.

Composition

- We initialize member objects explicitly through member initializers. This eliminates the overhead of doubly initializing member objects - once when the member object's default constructor is called and again when set functions are called in the constructor body (or later) to initialize the member object.
- A compilation error occurs if a member object is not initialized with a member initializer and the member object's class does not provide a default constructor (i.e. the member object's class defines one or more constructors, but none is a default constructor).

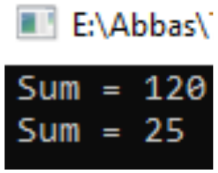
Example 1

```
class X {
    int d;
  public:
    void set_value(int k) {d = k;}
    void show_sum(int n){cout<<" Sum = "<< d+n <<endl;}
};

class Y {
  public:
    X a;
    void print_result() { a.show_sum(5);}
};

int main(){
   Y b;
   b.a.set_value(20);
   b.a.show_sum(100);
   b.print_result();

return 0;
}
```



Example 2

```
class A{
    int a;
  public:
    A(int a1)
      {cout<<" - Class A Costructor - "<<endl; a = a1;}
    void print()
      {cout<<" Class A with Data Member = "<<a<<endl;}
 };

class B{
    int b;
    A obj1;
  public:
    B(int x, int y): obj1(y)
      {cout<<" - Class B Costructor - "<<endl; b = x;}
    void print()
      {cout<<" Class B with Data Member = "<<b<<endl; obj1.print();}
 };

int main(){
   B obj2(1,5); obj2.print();

return 0;
}
```

E:\Abbas\Teaching\OOP\Lectures\Class

```
- Class A Costructor -
- Class B Costructor -
Class B with Data Member = 1
Class A with Data Member = 5
```

Example 3

```
class Date{
    int day,month,year;
  public:
    Date(int d = 1, int m = 1, int y = 1)
      {day = d; month = m; year = y;}
    void print()
      {cout<<day<<'/'<<month<<'/'<<year<<endl;}
};

class Student{
    char name[15];
    Date birthDate;
  public:
    Student(char* n, int bDay, int bMonth, int bYear):
        birthDate(bDay,bMonth,bYear){strcpy(name,n);}
    void print(){
        cout<<" Student Name = "<<name<<endl;
        cout<<" Birth Date = "; birthDate.print();
      }
};

int main(){
   Student s(" Muslim", 4, 2, 2006); s.print();
return 0;
}
```

```
E:\Abbas\Teaching\OOP\Lectu
Student Name =  Muslim
Birth Date = 4/2/2006
```

Example 4

```
class Time{
    int hour,minute;
  public:
    Time(int h = 1, int m = 1)
      {hour = h; minute = m;}
    void print()
      {cout<<hour<<':'<<minute<<endl;}
 };

class Exam{
    char name[15];
    Time startTime, endTime;
  public:
    Exam(char* n, int sHr, int sMin, int eHr, int eMin):
      startTime(sHr,sMin), endTime(eHr,eMin){strcpy(name, n);}
    void print(){
      cout<<" Exam = "<<name<<endl;
      cout<<" Starting Time = "; startTime.print();
      cout<<" Ending Time = "; endTime.print();
    }
 };
int main(){
   Exam e(" OOP ",10, 30, 12, 30); e.print();
return 0;
}
```

E:\Abbas\Teaching\OOP\Lec

```
Exam =  OOP
Starting Time = 10:30
Ending Time = 12:30
```

Conclusions

In this lecture we discussed about

- Association
- Aggregation
- Composition